Web Scraping Project Report

Project Title

Zomato Restaurant Data Scraping using Selenium

Student Details

Field	Details
Name	Taniya
Roll No.	8524255
Branch	B.Tech CSE (AI & ML)
College	JMIETI
Company	CodroidHub .Pvt.Ltd
Instructor	Devashish Kumar

Introduction

Zomato Restaurant Data Scraping using Selenium is a dynamic web scraping project developed using Python and

Selenium WebDriver. The main objective of this project is to automatically collect restaurant information from the Zomato website, which loads its content dynamically using JavaScript.

The scraper opens the Zomato NCR Restaurants webpage in a Chrome browser, scrolls through the page to load additional restaurant listings, and extracts important information such as the restaurant name, rating, and restaurant URL. The collected data is then organized into a structured format using a Pandas DataFrame and exported as a CSV file for future analysis.

This project demonstrates the practical implementation of browser automation, dynamic content handling, XPath-based element selection, data extraction, and data storage techniques. It provides a real-world example of how Selenium can be used for web scraping when traditional libraries like BeautifulSoup cannot access dynamically generated webpage content.

In [1]:
```
!pip install selenium webdriver-manager
```

```
Requirement already satisfied: selenium in c:\users\hp\anaconda3\lib\site-packages (4.45.0)
Requirement already satisfied: webdriver-manager in c:\users\hp\anaconda3\lib\site-packages (4.1.2)
Requirement already satisfied: certifi>=2026.2.25 in c:\users\hp\anaconda3\lib\site-packages (from selenium) (202
6.6.17)
Requirement already satisfied: trio<1.0,>=0.31.0 in c:\users\hp\anaconda3\lib\site-packages (from selenium) (0.33.0)
Requirement already satisfied: trio-websocket<1.0,>=0.12.2 in c:\users\hp\anaconda3\lib\site-packages (from selenium)
(0.12.2)
Requirement already satisfied: typing_extensions<5.0,>=4.15.0 in c:\users\hp\anaconda3\lib\site-packages (from seleni
um) (4.15.0)
Requirement already satisfied: urllib3<3.0,>=2.6.3 in c:\users\hp\anaconda3\lib\site-packages (from urllib3[socks]<
3.0,>=2.6.3->selenium) (2.7.0)
Requirement already satisfied: websocket-client<2.0,>=1.8.0 in c:\users\hp\anaconda3\lib\site-packages (from seleniu
m) (1.8.0)
Requirement already satisfied: attrs>=23.2.0 in c:\users\hp\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->sele
nium) (25.4.0)
Requirement already satisfied: sortedcontainers in c:\users\hp\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->s
elenium) (2.4.0)
Requirement already satisfied: idna in c:\users\hp\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium)
(3.11)
Requirement already satisfied: outcome in c:\users\hp\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium)
(1.3.0.post0)
Requirement already satisfied: sniffio>=1.3.0 in c:\users\hp\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->sel
enium) (1.3.0)
Requirement already satisfied: cffi>=1.14 in c:\users\hp\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->seleniu
m) (2.0.0)
Requirement already satisfied: wsproto>=0.14 in c:\users\hp\anaconda3\lib\site-packages (from trio-websocket<1.0,>=
0.12.2->selenium) (1.3.2)
Requirement already satisfied: pysocks!=1.5.7,<2.0,>=1.5.6 in c:\users\hp\anaconda3\lib\site-packages (from urllib3[s
ocks]<3.0,>=2.6.3->selenium) (1.7.1)
Requirement already satisfied: requests in c:\users\hp\appdata\roaming\python\python313\site-packages (from webdrive
r-manager) (2.34.2)
Requirement already satisfied: python-dotenv in c:\users\hp\anaconda3\lib\site-packages (from webdriver-manager)
(1.1.0)
Requirement already satisfied: packaging in c:\users\hp\anaconda3\lib\site-packages (from webdriver-manager) (25.0)
Requirement already satisfied: pycparser in c:\users\hp\anaconda3\lib\site-packages (from cffi>=1.14->trio<1.0,>=0.3
1.0->selenium) (2.23)
Requirement already satisfied: h11<1,>=0.16.0 in c:\users\hp\anaconda3\lib\site-packages (from wsproto>=0.14->trio-we
bsocket<1.0,>=0.12.2->selenium) (0.16.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\hp\anaconda3\lib\site-packages (from requests->we
bdriver-manager) (3.4.4)
```

Imported Libraries

The following Python libraries were imported to perform dynamic web scraping, browser automation, data extraction, and data processing in this project.

Library	Purpose
selenium	Automates browser actions and interacts with web elements.
webdriver_manager	Automatically downloads and manages the ChromeDriver.
time	Adds delays to allow dynamic content to load properly.
re	Uses regular expressions to extract restaurant ratings.
pandas	Stores scraped data in a DataFrame and exports it to a CSV file.

In [2]:
```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time
import re
```

```
import pandas as pd
```

Step 1: Create the Web Scraping Function

In [8]:
```
def zomato_scraper():

    options = webdriver.ChromeOptions()
    options.add_argument("--start-maximized")

    driver = webdriver.Chrome(
        service=Service(ChromeDriverManager().install()),
        options=options
    )

    driver.get("https://www.zomato.com/ncr/restaurants")

    time.sleep(10)

    # Scroll for dynamic loading
    for _ in range(6):
        driver.execute_script("window.scrollBy(0,3000)")
        time.sleep(2)

    data = []

    cards = driver.find_elements(By.XPATH, "//a[contains(@href,'/ncr/')]")

    print("Restaurants Found :", len(cards))

    for c in cards:

        try:

            link = c.get_attribute("href")

            if not link:
                continue

            text = c.text.strip()
```

```
            if not text:
                continue

            lines = text.split("\n")

            name = lines[0] if len(lines) > 0 else "N/A"

            rating = "N/A"

            match = re.search(r"\b\d\.\d\b", text)

            if match:
                rating = match.group()

            data.append({
                "Restaurant Name": name,
                "Rating": rating,
                "Restaurant Link": link
            })

        except:
            pass

    driver.quit()

    return data
```

Step 2: Execute the Scraper

In [9]:
```
restaurants = zomato_scraper()

print("Total Restaurants :", len(restaurants))
```

```
Restaurants Found : 385
Total Restaurants : 239
```

Step 3: Convert Data into a DataFrame

In [10]:
```
df = pd.DataFrame(restaurants)
```

```
df.head(10)
```

Out[10]:

	Restaurant Name	Rating	Restaurant Link
0	All collections in Delhi NCR	N/A	https://www.zomato.com/ncr/collections
1	Insta-worthy spots	N/A	https://www.zomato.com/ncr/insta-worthy
2	Omakase bars	N/A	https://www.zomato.com/ncr/omakase-bars
3	Secret speakeasy bars	N/A	https://www.zomato.com/ncr/secret-speakeasy-bars
4	Must visit cafes	N/A	https://www.zomato.com/ncr/great-cafes
5	Promoted	N/A	https://www.zomato.com/ncr/lord-of-the-drinks-...
6	Lord Of The Drinks	4.3	https://www.zomato.com/ncr/lord-of-the-drinks-...
7	Promoted	N/A	https://www.zomato.com/ncr/plum-coffee-and-coc...
8	Plum Coffee and Cocktails	4.5	https://www.zomato.com/ncr/plum-coffee-and-coc...
9	Promoted	N/A	https://www.zomato.com/ncr/farzi-cafe-connaugh...

Step 4: Display the Complete Dataset

In [11]:
```
df
```

Out[11]:

	Restaurant Name	Rating	Restaurant Link
0	All collections in Delhi NCR	N/A	https://www.zomato.com/ncr/collections
1	Insta-worthy spots	N/A	https://www.zomato.com/ncr/insta-worthy
2	Omakase bars	N/A	https://www.zomato.com/ncr/omakase-bars
3	Secret speakeasy bars	N/A	https://www.zomato.com/ncr/secret-speakeasy-bars
4	Must visit cafes	N/A	https://www.zomato.com/ncr/great-cafes
...	...	...	...
234	Koca	4.0	https://www.zomato.com/ncr/koca-dlf-phase-5-gu...
235	Promoted	N/A	https://www.zomato.com/ncr/nod-punjabi-bagh-ne...
236	NOD	3.8	https://www.zomato.com/ncr/nod-punjabi-bagh-ne...
237	Flat 20% OFF	N/A	https://www.zomato.com/ncr/dearie-bar-brewery-...
238	Dearie - Bar. Brewery. Kitchen.	4.3	https://www.zomato.com/ncr/dearie-bar-brewery-...

239 rows × 3 columns

Step 5: Save the Data as a CSV File

In [12]:
```
df.to_csv("zomato_restaurants.csv", index=False)

print("CSV Saved Successfully")
```

```
CSV Saved Successfully
```

Conclusion

The **Zomato Restaurant Data Scraping using Selenium** project successfully demonstrated how dynamic web pages

can be scraped using Python and Selenium WebDriver. The scraper was able to automate browser interactions, scroll through dynamically loaded content, and extract valuable restaurant information such as the restaurant name, rating, and URL.

The extracted data was organized into a structured Pandas DataFrame and saved as a CSV file, making it suitable for further analysis, visualization, or machine learning applications. This project highlights the effectiveness of Selenium for scraping JavaScript-based websites where traditional HTML parsing libraries may not be sufficient.

Overall, this project enhanced practical knowledge of web scraping, browser automation, data extraction, data preprocessing, and data storage. It also provided valuable hands-on experience in working with real-world web data and modern Python libraries.